# Общая организация проекта и тестирование

## Лекция №8

Николай Андреевич Вениаминов

Политех

15 октября 2026 г.



# План на сегодня

1. Общая структура проекта
2. Тестирование

# План на сегодня

1. Общая структура проекта
2. Тестирование

# Структура проекта

Приведенная ниже структура не является единственно правильной или канонической.
Полезно придерживаться фиксированной структуры проекта, особенно при работе в команде.

- `data` – для данных (исключается из системы контроля версий!)
  - `raw` – исходные данные, не должны меняться кодом
  - `processed` – переработанные данные (как правило промежуточные результаты)
- `docs` – документация проекта, например, сделанная с помощью Sphinx
- `models` – модели (исключается из системы контроля версий для тяжелых моделей)
- `notebooks` – Jupyter notebook'и
- `scripts` – исполняемые программы (скрипты), при этом не обязательно только `.py`, но также и `.sh`
- `src` – исходный код пакетов и модулей
- `tests` – тесты
- Верхнеуровневые Markdown файлы проекта – `README.md` и `CONTRIBUTING.md`
- Файлы управления зависимостями, такие как `pyproject.toml`, `poetry.lock`, `uv.lock`, `requirements.txt`
- Файлы управления сборкой и развертыванием, например, `.gitlab-ci.yml`

# Шаблоны для проектов: cookiecutter

## Определение

**Cookiecutter** – утилита командной строки, позволяющая создавать заготовки проектов на основании шаблона.

Шаблон проекта представляет из себя струтуру директорий и файлов, параметрируемых через Jinja template'ы.
Особенности:

1. Есть множество общедоступных шаблонов
2. Можно создать свой собственный шаблон
3. Поддерживает пред- и пост-обработку (например, код сразу может быть опубликован в репозитории)

# План на сегодня

1. Общая структура проекта
2. Тестирование

# Тестирование: виды

1. **По объектам тестирования**

   **Функциональное тестирование** (проверка соответствия функциональности требованиям):

   - *Unit-тестирование* (модульное) — проверка отдельных компонентов в изоляции.
   - *Интеграционное* — проверка взаимодействия между модулями.
   - *Системное* — проверка всей системы в целом.
   - *Приёмочное* — финальная проверка перед передачей заказчику.

   **Нефункциональное тестирование** (проверка характеристик, не связанных напрямую с функционалом):

   - *Производительность* (нагрузочное, стресс-тестирование, тестирование пиковых нагрузок, тестирование на выносливость, тестирование масштабируемости).
   - *Безопасность* — проверка защиты от несанкционированного доступа.
   - *Совместимость* — работа на разных устройствах, ОС, браузерах.
   - *Отказоустойчивость* — поведение при сбоях внешних систем.
   - *Восстановление* — скорость возврата к работе после сбоя.
   - *Интерфейс* — корректность работы пользовательского интерфейса.

# Тестирование: виды

2. **По характеру сценариев**
   - **Позитивное тестирование** — проверка работы программы в штатных условиях (корректный ввод данных, стандартные операции).
   - **Негативное тестирование** — проверка на некорректные данные или нештатные ситуации.
3. **По степени автоматизации**
   - **Ручное тестирование** — выполнение тестов человеком без автоматизированных инструментов.
   - **Автоматизированное тестирование** — использование специальных инструментов для автоматического выполнения тестов.
4. **По степени знания системы**
   - **«Чёрный ящик»** — тестирование интерфейса без доступа к коду.
   - **«Белый ящик»** — анализ внутренней структуры и кода.
   - **«Серый ящик»** — комбинация обоих подходов (частичный доступ к коду).
5. **По времени проведения**
   - **Альфа-тестирование** — внутреннее тестирование внутри компании-разработчика.
   - **Дымовое тестирование** — быстрая проверка после изменений в коде.
   - **Регрессионное тестирование** — проверка «старых» функций после обновлений.
   - **Приёмочное тестирование** — финальная проверка заказчиком.
   - **Бета-тестирование** — привлечение реальных пользователей к тестированию почти готового продукта.

# Тестирование: pytest

- Тесты организуются в модули, классы и функции с префиксами `Test` или `test`; все тесты находятся автоматически
- Основной шаблон, используемый для проверки – `assert`
- `pytest.approx` позволяет проверять примерное равенство, что удобно для чисел с плавающей точкой
- `pytest.raises` позволяет тестировать вызов исключений
- `pytest.fixture` создает переиспользуемые объекты для использования в параметрированных тестах

# Задача №6: повороты в пространстве через кватернионы

**Цель:** реализовать класс `Quaternion` и использовать его для поворота 3D-векторов

**Зачем это нужно:**

- Избегание *gimbal lock* (в отличие от углов Эйлера)
- Эффективная композиция поворотов через умножение кватернионов
- Стандарт в робототехнике, графике, симуляциях и задачах ориентации

**Что требуется:**

- Реализовать класс с арифметикой кватернионов (сложение, умножение, сопряжение, норма)
- Реализовать поворот вектора по формуле $qvq^{-1}$
- Написать тесты на корректность поворотов и композиции

Примечание: внешние библиотеки для самой математики кватернионов использовать нельзя; `pytest` и `math` — можно.

*Название репозитория: task 6: quaternions*